

# Hadoop YARN Scheduler 架构设计文档

核心架构设计 · 调度器原理 · 设计模式 · 流程图 · 时序图 · 调用链

模块：hadoop-yarn-server-resourcemanager / scheduler

版本：release-3.3.5-RC0

生成工具：AI Architecture Analyzer

生成日期：2026年03月26日

# 第1章 模块概述

## 1.1 模块定位

YARN (Yet Another Resource Negotiator) 是 Hadoop 生态中的资源管理与作业调度平台。scheduler 模块是 YARN ResourceManager 的核心子系统，负责将集群计算资源（CPU、内存等）按照特定策略分配给各个应用程序。该模块实现了多种调度策略，包括容量调度器（Capacity Scheduler）、公平调度器（Fair Scheduler）和先进先出调度器（FIFO Scheduler），以满足不同多租户场景下的资源管理需求。

本文档基于 Hadoop release-3.3.5-RC0 版本，对 scheduler 模块下的 306 个 Java 源文件进行了全量分析，覆盖 22 个子目录，深入解析三款调度器的架构设计、调度原理、设计模式与关键调用链。

## 1.2 核心目标

- 多租户资源隔离：通过层级队列实现不同租户/组织间的资源隔离和按比例分配
- 资源利用率最大化：支持弹性资源共享、队列间资源借用与回收
- 调度公平性保障：通过公平份额算法、抢占机制确保各应用获得合理份额
- 数据局部性优化：通过延迟调度机制优先将任务调度到数据所在节点
- 可插拔与可扩展：基于接口-抽象类-实现类的层次设计，支持自定义调度策略

## 1.3 模块文件统计

| 子目录              | 文件数 | 核心职责          |
|------------------|-----|---------------|
| scheduler/ (根目录) | 38  | 调度器核心接口、基类、工具 |
| capacity/        | 99  | 容量调度器及其子模块    |
| fair/            | 55  | 公平调度器及其子模块    |
| fifo/            | 2   | 先进先出调度器       |
| distributed/     | 4   | 分布式调度/机会容器    |
| event/           | 17  | 调度事件定义        |
| policy/          | 12  | 应用排序策略        |
| placement/       | 13  | 队列放置规则        |
| activities/      | 13  | 调度活动记录        |
| common/          | 11  | 公共数据结构        |
| constraint/      | 34  | 放置约束处理        |
| 其他               | 8   | 容器预留、Node属性等  |



## 第2章 核心架构设计

### 2.1 调度器接口继承体系

YARN 调度器采用分层接口设计，从事件处理基础接口逐层扩展到具体调度器实现。这种设计遵循了接口隔离原则（ISP）和依赖倒置原则（DIP），使得 ResourceManager 无需关心调度器的具体实现。

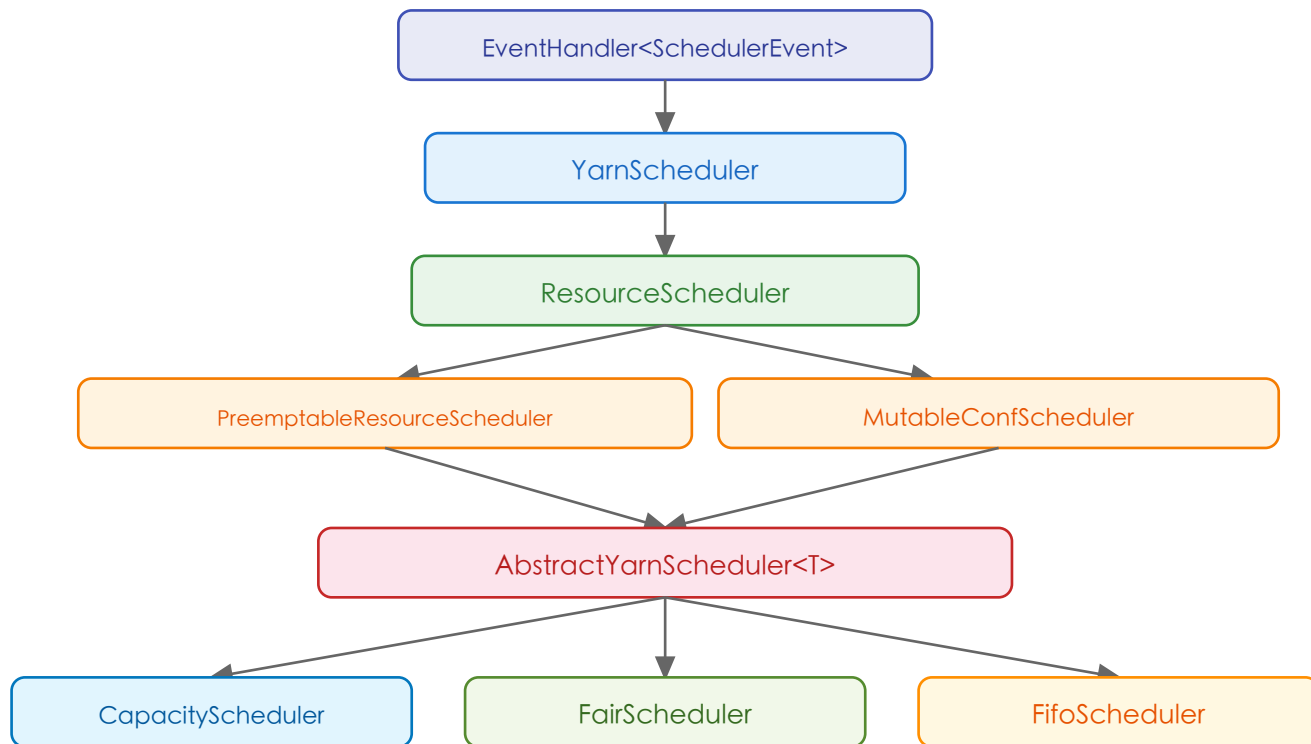


图2-1 调度器接口继承体系

各层接口职责：

- `EventHandler`：事件驱动基础，处理 `NODE_ADDED/REMOVED`、`APP_ADDED/REMOVED` 等 12 种调度事件
- `YarnScheduler`：顶层接口，定义 `allocate()`（心跳分配）、`getQueueInfo()`、`getNodeReport()` 等核心 API
- `ResourceScheduler`：核心资源调度接口，扩展 `Recoverable` 支持 RM 故障恢复
- `PreemptableResourceScheduler`：抢占接口，定义 `killContainer()` 方法
- `MutableConfScheduler`：动态配置接口，支持运行时修改队列容量而无需重启
- `AbstractYarnScheduler`：抽象基类（约1400行），提供节点管理/应用管理/容器恢复公共骨架

## 2.2 整体分层架构

scheduler 模块整体采用四层架构：接口层、抽象层、调度策略层和支撑服务层。

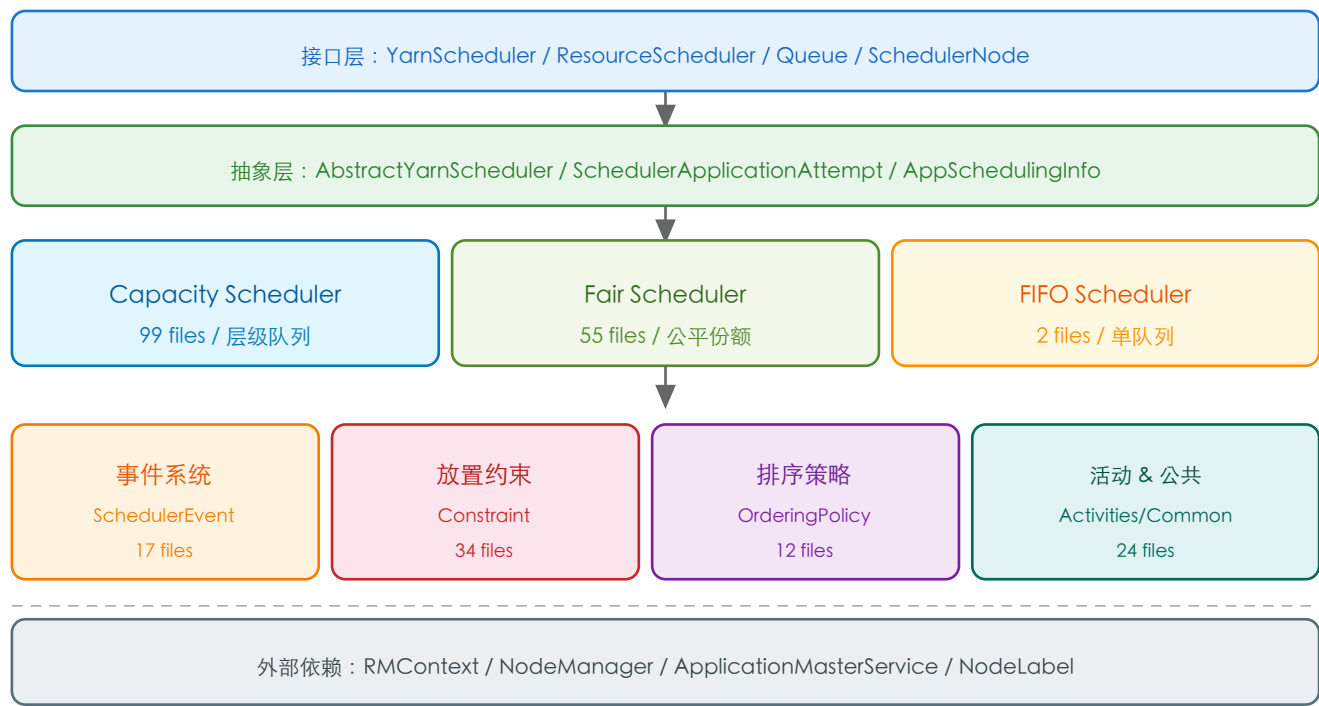


图2-2 scheduler 模块四层架构图

## 2.3 核心接口与抽象类清单

| 类名                           | 类型  | 行数    | 职责                                       |
|------------------------------|-----|-------|------------------------------------------|
| YarnScheduler                | 接口  | ~418  | 调度器顶层 API : allocate / getQueueInfo      |
| ResourceScheduler            | 接口  | ~78   | 核心资源调度 : reinitialize / setRMContext     |
| PreemptableResourceScheduler | 接口  | ~52   | 抢占接口 : killContainer                     |
| MutableConfScheduler         | 接口  | ~53   | 动态配置 : 运行时修改队列容量                         |
| AbstractYarnScheduler        | 抽象类 | ~1400 | 骨架实现 : 节点/应用管理/容器恢复                      |
| Queue                        | 接口  | ~141  | 队列抽象 : getQueueInfo / getQueueUserAcInfo |
| SchedulerNode                | 抽象类 | ~485  | 调度节点 : 容器分配/预留/资源跟踪                      |
| SchedulerApplicationAttempt  | 抽象类 | ~1600 | 应用尝试 : 容器管理/资源请求/黑名单                     |
| AppSchedulingInfo            | 类   | ~700  | 应用调度信息 : ResourceRequest 管理              |
| ClusterNodeTracker           | 类   | ~522  | 集群节点跟踪 : 添加/移除/查询                        |

# 第3章 Capacity Scheduler（容量调度器）详解

## 3.1 概述与定位

Capacity Scheduler（容量调度器）是 YARN 的默认调度器，是生产环境中使用最广泛的调度策略。其设计目标是在保证每个租户最小资源保障的前提下，通过弹性资源共享最大化集群利用率。模块包含 99 个 Java 文件，是 scheduler 中最复杂的子系统。

## 3.2 队列层级模型

Capacity Scheduler 基于层级队列（Hierarchical Queues）模型。队列形成树形结构，root 队列为根节点，支持任意深度嵌套。分为 ParentQueue（父队列）和 LeafQueue（叶队列，接收应用提交）。

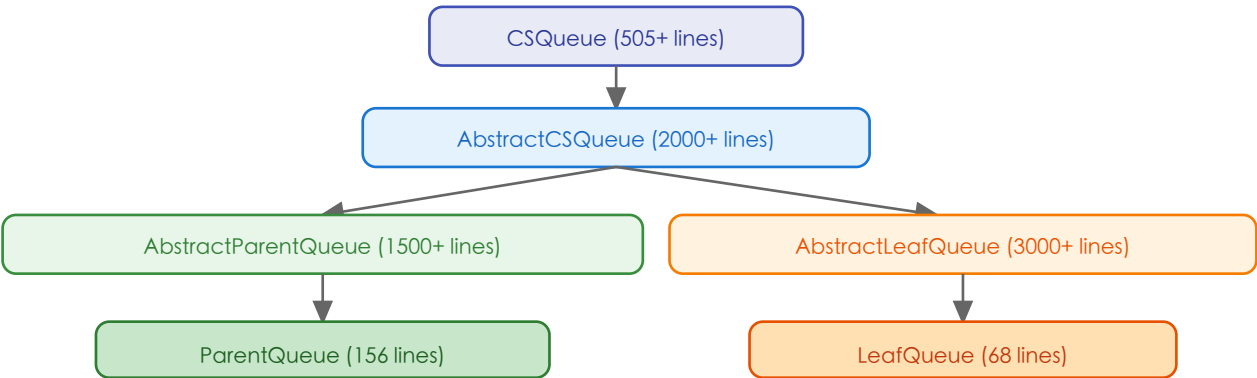


图3-1 CS 队列类继承体系（薄子类 + 厚抽象类设计）

关键设计特点：采用「厚抽象类 + 薄子类」设计模式。AbstractCSQueue（~2000行）、AbstractParentQueue（~1500行）、AbstractLeafQueue（~3000行）承载了绝大部分逻辑，具体子类 ParentQueue（156行）和 LeafQueue（68行）仅做极少量定制。

## 3.3 三种容量配置模式

| 模式   | 配置方式                                 | 说明                |
|------|--------------------------------------|-------------------|
| 百分比  | root.a.capacity=60                   | 按百分比分配，同级队列之和100% |
| 权重   | root.a.capacity=w:6                  | 按权重比例分配，更灵活       |
| 绝对资源 | root.a.capacity=[mem=10240,vcores=8] | 指定绝对资源量           |

## 3.4 核心组件

| 组件    | 类名                                   | 职责             |
|-------|--------------------------------------|----------------|
| 调度器主类 | CapacityScheduler                    | 入口：心跳/事件/容器分配  |
| 队列管理  | CSQueueManager                       | 队列CRUD、权限、动态创建 |
| 配置管理  | CSConfiguration                      | 配置解析：容量/ACL/标签 |
| 容器分配器 | RegularContainerAllocator            | 两阶段分配：选队列->分配  |
| 异步调度  | AsyncSchedulingMonitor               | 独立线程异步调度       |
| 抢占监控  | ProportionalCapacityPreemptionPolicy | 资源使用监控/容器抢占    |

### 3.5 心跳调度核心流程

NodeManager 通过心跳向 RM 报告节点状态，触发 CapacityScheduler.nodeUpdate() 进行调度：

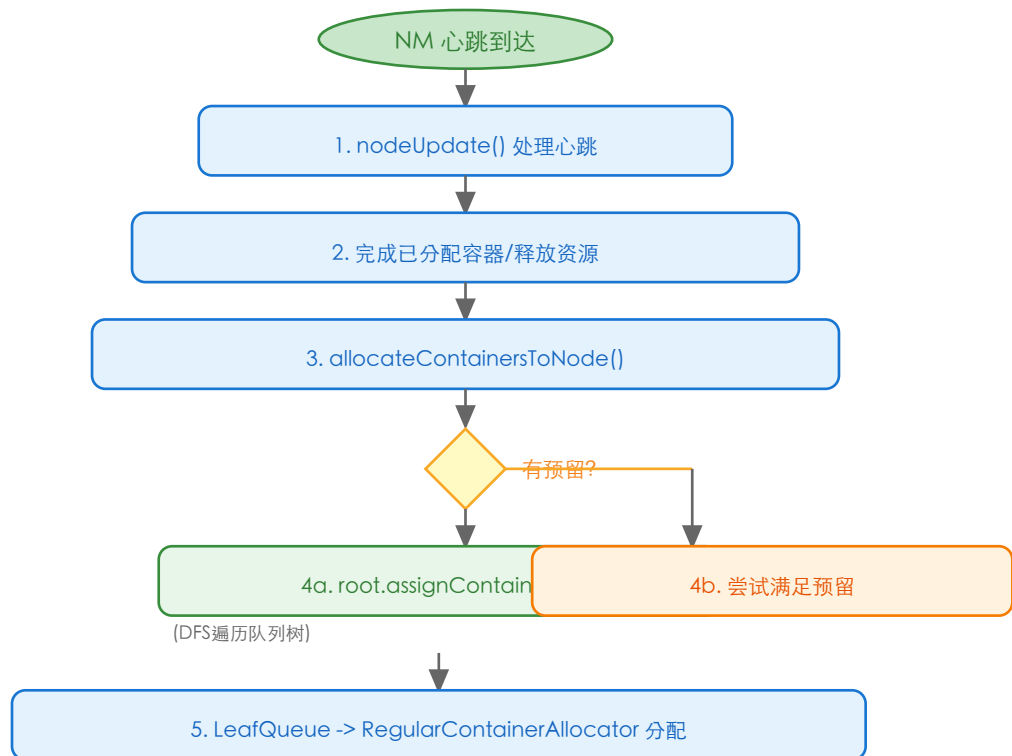


图3-2 Capacity Scheduler 心跳调度核心流程

### 3.6 延迟调度与资源预留

延迟调度：为优化数据局部性，当资源请求指定节点/机架偏好时，调度器等待一定错过次数后逐级降级：NODE\_LOCAL -> RACK\_LOCAL -> OFF\_SWITCH。核心参数为 node-locality-delay 和 rack-locality-additional-delay。

资源预留：当节点资源不足以分配请求的容器时，调度器在该节点上创建预留（Reservation），等待资源释放后优先满足。预留容器在下次心跳时优先检查。

### 3.7 资源抢占机制

ProportionalCapacityPreemptionPolicy 定期检查各队列资源使用：

- 计算每个队列的「理想分配量」和「实际使用量」
- 对资源超用的队列标记待抢占容器（最近启动的优先）
- 通过 killContainer() 终止选中的容器，释放资源给饥饿队列
- 支持自然到期模式（等容器完成而非强制杀死）

# 第4章 Fair Scheduler（公平调度器）详解

## 4.1 概述与定位

Fair Scheduler Scheduler 的核心目标是确保所有应用随时间推移获得大致相等的资源份额。通过动态计算公平份额（Fair Share），让空闲资源流向需要的应用，实现「按需分配、动态均衡」。模块包含 55 个 Java 文件。

## 4.2 队列模型与 Schedulable 接口

Fair Scheduler 引入 Schedulable 接口作为可调度实体的统一抽象。FSQueue（队列）和 FSAppAttempt（应用尝试）都实现 Schedulable，调度算法可统一处理队列间和应用间分配（组合模式）。

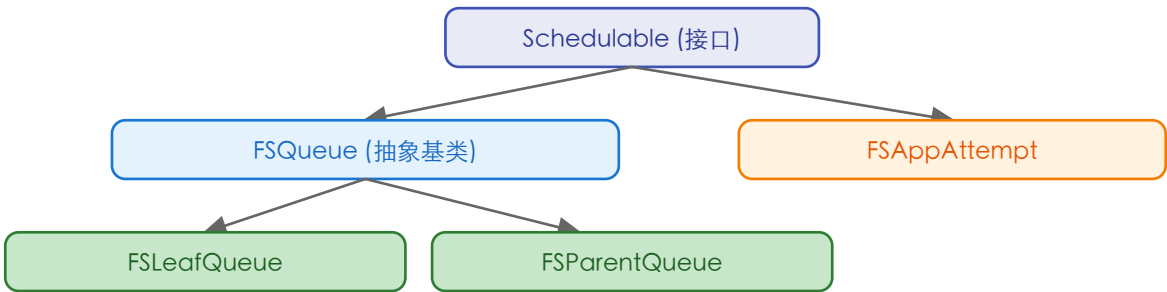


图4-1 Fair Scheduler 队列类继承体系

## 4.3 公平份额计算算法（ComputeFairShares）

核心算法位于 ComputeFairShares 类（249行），采用二分查找计算加权公平份额：

- 输入：一组 Schedulable 实体及其权重（weight）和需求量（demand），以及总可用资源
- 目标：找到阈值 R，使每个实体份额 = min(demand, weight \* R)，所有份额之和 = 总资源
- 方法：在 [0, maxResource] 范围二分查找，迭代 25 次获得足够精度
- 特殊处理：总需求 <= 总资源时，每个实体直接获得全部需求量

## 4.4 三种调度策略（策略模式）

| 策略   | 实现类                            | 排序规则                  | 适用场景    |
|------|--------------------------------|-----------------------|---------|
| Fair | FairSharePolicy                | (usage/weight) 比值小者优先 | 默认，资源均分 |
| DRF  | DominantResourceFairnessPolicy | 主导资源使用率小者优先           | 混合负载    |
| FIFO | FifoPolicy                     | 按提交时间排序               | 兼容老行为   |

## 4.5 配置热加载与抢占

配置热加载：AllocationFileLoaderService

周期性检查

fair-scheduler.xml

修改时间（默认10秒），变更时自动重新加载配置并更新运行时队列和权重。

抢占机制基于两种饥饿检测：

- 公平份额饥饿：使用量  $< \text{fairShare} * \text{threshold}$ ，超过 timeout 后触发抢占
- 最小份额饥饿：使用量  $< \text{minShare}$ ，超过 minShareTimeout 后触发抢占

# 第5章 FIFO Scheduler 与分布式调度

## 5.1 FIFO Scheduler

FifoScheduler 是最简单的调度器，仅 2 个文件。单队列 "default"，按提交顺序先到先得。不支持资源隔离和抢占，适用于开发测试或单用户集群。

## 5.2 分布式调度（Distributed Scheduling）

面向机会容器（Opportunistic Containers）的分配。传统容器是「保证容器」需严格预留资源；机会容器可在节点资源未被完全占用时执行，提高利用率。

| 类                                          | 职责                  |
|--------------------------------------------|---------------------|
| CentralizedOpportunisticContainerAllocator | 集中式机会容器分配器          |
| ClusterNode                                | 集群节点信息封装            |
| NodeQueueLoadMonitor                       | 节点队列负载监控（最小堆/最大堆排序） |
| QueueLimitCalculator                       | 队列限额计算器             |

## 第6章 核心设计模式详解

---

### 6.1 模板方法模式 (Template Method)

体现类：AbstractYarnScheduler / AbstractCSQueue / AbstractParentQueue / AbstractLeafQueue

整个调度器框架的核心骨架。AbstractYarnScheduler

定义通用算法骨架（初始化、节点管理、应用生命周期），子类只需重写

allocate()、nodeUpdate() 等方法。CS 队列的 AbstractParentQueue.assignContainers() 定义 DFS 遍历子队列框架，AbstractLeafQueue.assignContainers() 定义遍历应用并尝试分配的框架。

### 6.2 策略模式 (Strategy)

体现类：SchedulingPolicy / FairSharePolicy / DominantResourceFairnessPolicy / FifoPolicy

Fair Scheduler 中最优雅的设计。SchedulingPolicy 定义 getComparator() 和 computeShares() 两个抽象方法，三种策略各自提供不同算法。队列可运行时切换策略。类似地，CS 中的 OrderingPolicy 接口也是策略模式的应用。

### 6.3 观察者/事件驱动模式 (Observer / Event-Driven)

体现类：EventHandler / SchedulerEventType / 各 SchedulerEvent 子类

调度器采用事件驱动架构。SchedulerEventType 定义 12 种事件类型，调度器实现 EventHandler.handle() 作为统一入口，按类型分发处理。RM 与调度器通过事件解耦。

### 6.4 组合模式 (Composite)

体现类：CSQueue / ParentQueue / LeafQueue、FSQueue / FSParentQueue / FSLeafQueue

队列树形结构是经典组合模式。ParentQueue 递归分发 assignContainers()，LeafQueue 执行实际分配。Schedulable 接口更进一步让队列和应用都成为可调度实体。

### 6.5 工厂方法模式 (Factory Method)

体现类：SchedulingPolicy.getInstance() / PlacementRule 工厂

SchedulingPolicy 通过 getInstance(clazz) 动态加载策略实例。PlacementRule 根据配置创建不同放置规则（UserPlacementRule、SpecifiedPlacementRule 等）。

### 6.6 责任链模式 (Chain of Responsibility)

体现类：PlacementRule 链 / placement/ 目录

Fair Scheduler 队列放置采用责任链：多个 PlacementRule 组成链表，应用提交时依次经过各规则。每个规则可返回目标队列或跳过传递给下一个。包括

UserPlacementRule 、 PrimaryGroupPlacementRule 、 SpecifiedPlacementRule 等。

## 第7章 核心流程图

---

### 7.1 应用提交与队列放置流程

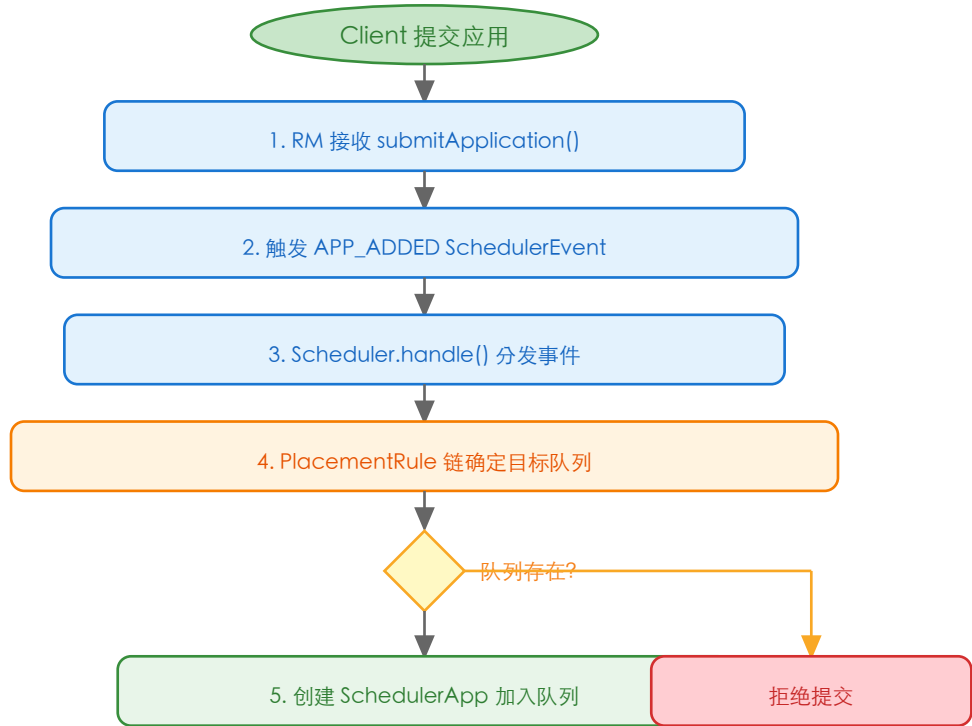


图7-1 应用提交与队列放置流程

## 7.2 Fair Scheduler 公平份额更新流程

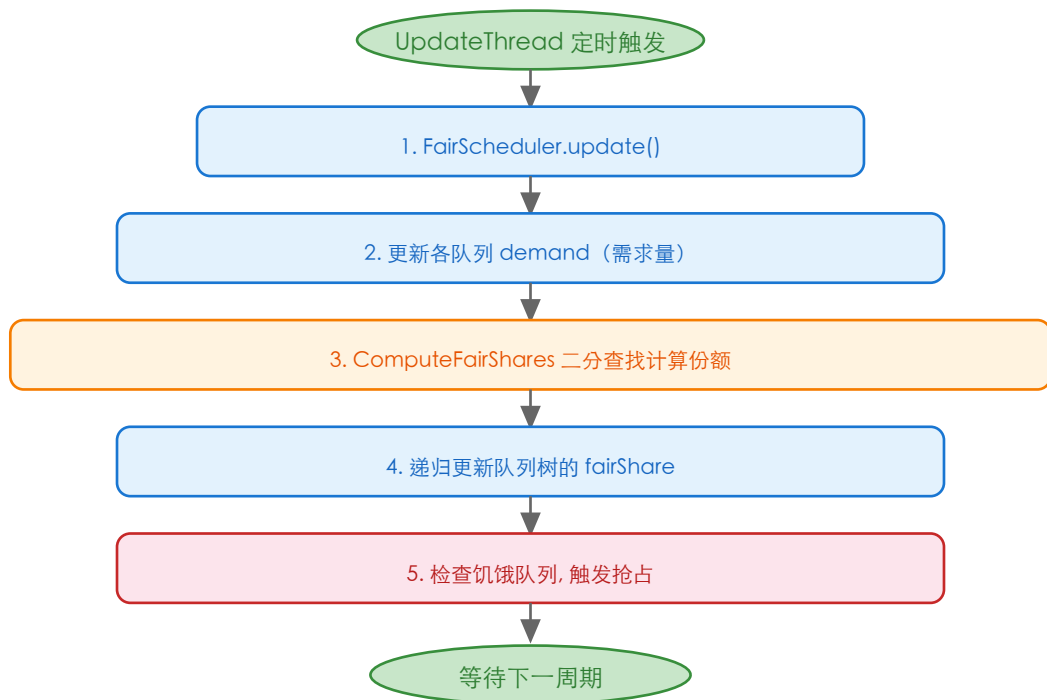


图7-2 Fair Scheduler 公平份额更新流程

## 第8章 核心时序图

### 8.1 Capacity Scheduler 容器分配时序图

当 NM 心跳到达时，CapacityScheduler 触发容器分配，涉及多个组件协作：

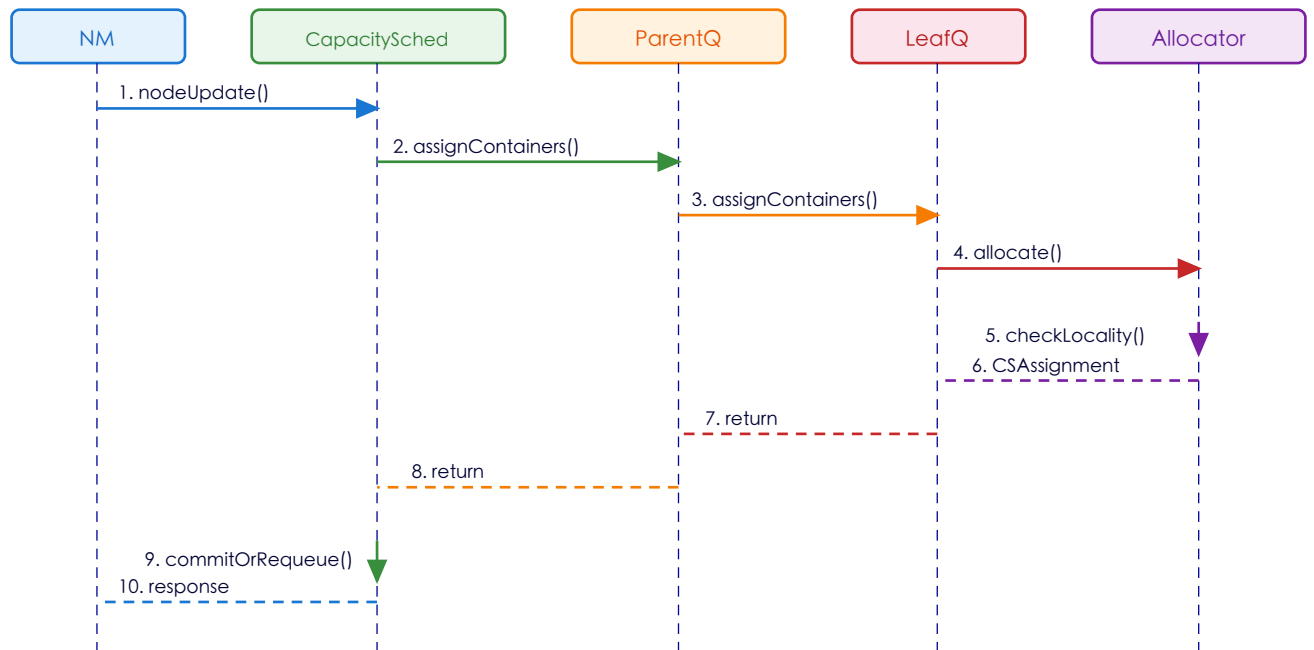


图8-1 Capacity Scheduler 容器分配时序图

## 8.2 Fair Scheduler 应用调度时序图

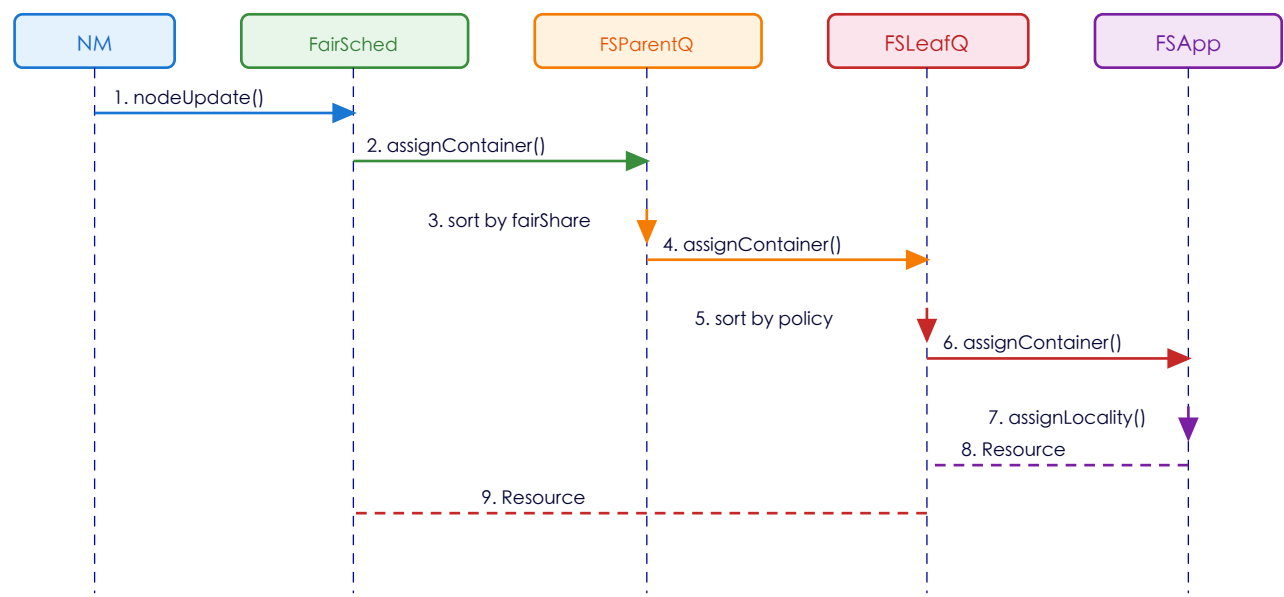


图8-2 Fair Scheduler 应用调度时序图

## 第9章 关键场景调用链

### 9.1 场景一：CS 容器分配完整调用链

触发条件：NodeManager 心跳到达，节点有可用资源

| 步骤 | 类.方法                                              | 说明                        |
|----|---------------------------------------------------|---------------------------|
| 1  | CapacityScheduler.nodeUpdate()                    | 处理 NODE_UPDATE 事件，释放已完成容器 |
| 2  | CS.allocateContainersToNode()                     | 检查节点预留，决定分配入口             |
| 3  | ParentQueue.assignContainers()                    | DFS 遍历子队列，按容量排序           |
| 4  | LeafQueue.assignContainers()                      | 按 OrderingPolicy 排序应用     |
| 5  | RegularContainerAllocator.assignContainers()<br>) | 匹配资源请求与节点                 |
| 6  | SchedulerNode.allocateContainer()                 | 在节点上分配容器                  |

### 9.2 场景二：FS 公平份额重计算

触发条件：UpdateThread 周期性触发（默认500ms）

| 步骤 | 类.方法                                    | 说明         |
|----|-----------------------------------------|------------|
| 1  | FairScheduler.update()                  | 触发入口       |
| 2  | FSQueue.updateDemand()                  | 递归更新各队列需求量 |
| 3  | ComputeFairShares.computeShares()       | 二分查找计算公平份额 |
| 4  | FSQueue.setFairShare()                  | 设置各队列的公平份额 |
| 5  | FairScheduler.preemptTasksIfNecessary() | 检查饥饿，触发抢占  |

### 9.3 场景三：队列动态创建（CS auto-queue）

触发条件：应用提交到不存在的队列，且父队列配置了 auto-queue-creation

| 步骤 | 类.方法                                       | 说明       |
|----|--------------------------------------------|----------|
| 1  | CapacityScheduler.handle(APP_ADDED)        | 收到应用添加事件 |
| 2  | CSQueueManager.getQueue()                  | 查找目标队列   |
| 3  | AutoCreatedQueueManager.createQueue()<br>) | 自动创建叶队列  |

| 步骤 | 类.方法                                   | 说明         |
|----|----------------------------------------|------------|
| 4  | AbstractCSQueue.initializeQueueState() | 初始化队列配置和容量 |
| 5  | CSQueueManager.addQueue()              | 注册到队列管理器   |

## 9.4 场景四：资源抢占执行

触发条件：抢占监控器检测到队列资源饥饿

| 步骤 | 类.方法                                                       | 说明      |
|----|------------------------------------------------------------|---------|
| 1  | ProportionalCapacityPreemptionPolicy.editSchedule()        | 计算抢占计划  |
| 2  | IntraQueuePreemptionComputePlugin.computeIdealAllocation() | 计算理想分配  |
| 3  | PreemptionManager.markContainersForPreemption()            | 标记待抢占容器 |
| 4  | CapacityScheduler.killContainer()                          | 终止选中的容器 |
| 5  | SchedulerNode.releaseContainer()                           | 释放节点资源  |

# 第10章 配置与扩展

## 10.1 Capacity Scheduler 关键配置

| 配置项                                                                     | 默认值               | 说明                  |
|-------------------------------------------------------------------------|-------------------|---------------------|
| yarn.resourcemanager.scheduler.class                                    | CapacityScheduler | 调度器实现类              |
| yarn.scheduler.capacity.root..capacity                                  | 100               | 队列容量（百分比/权重/绝对）     |
| yarn.scheduler.capacity.root..maximum-capacity                          | 100               | 队列最大容量              |
| yarn.scheduler.capacity.node-locality-delay                             | 40                | NODE_LOCAL 延迟调度错过次数 |
| yarn.scheduler.capacity.schedule-asynchronously.enable                  | false             | 启用异步调度              |
| yarn.scheduler.capacity.per-node-heartbeat.multiple-assignments-enabled | true              | 单次心跳多次分配            |

## 10.2 Fair Scheduler 关键配置

| 配置项                                    | 默认值                | 说明        |
|----------------------------------------|--------------------|-----------|
| yarn.resourcemanager.scheduler.class   | FairScheduler      | 调度器实现类    |
| yarn.scheduler.fair.allocation.file    | fair-scheduler.xml | 分配配置文件路径  |
| yarn.scheduler.fair.update-interval-ms | 500                | 公平份额更新间隔  |
| yarn.scheduler.fair.preemption         | false              | 启用抢占      |
| yarn.scheduler.fair.sizebasedweight    | false              | 基于应用大小的权重 |
| yarn.scheduler.fair.assignmultiple     | false              | 单次心跳多次分配  |

## 10.3 扩展点

- 自定义调度策略：继承 SchedulingPolicy 并实现 getComparator() 和 computeShares()
- 自定义放置规则：继承 PlacementRule 并实现 getQueueForApp() 方法
- 自定义排序策略：实现 OrderingPolicy 接口的 getAssignmentIterator()
- 自定义约束处理：实现 PlacementConstraintManager 中的约束处理器

# 第11章 设计亮点与总结

## 11.1 设计亮点

- 事件驱动解耦：调度器与 RM 通过事件系统解耦，12 种事件类型覆盖完整生命周期
- 厚抽象类设计：CS 通过 Abstract\*Queue 系列类将 6000+ 行逻辑集中在抽象层，子类极简
- 策略模式灵活性：Fair Scheduler 的三种调度策略可运行时切换，队列级独立配置
- 二分查找公平份额：ComputeFairShares 用 25 次迭代在  $O(N \cdot \log R)$  时间内完成份额计算
- 延迟调度优化：三级局部性降级机制平衡了数据局部性与调度吞吐量
- 异步调度提速：CS 支持 AsyncSchedulingMonitor 独立线程调度，不受心跳频率限制

## 11.2 设计局限

- CS 抽象基类过大：AbstractLeafQueue 约 3000 行，AbstractParentQueue 约 1500 行，维护难度高
- Fair Scheduler 维护状态：社区已标记 Fair Scheduler 为「不推荐使用」，建议迁移到 CS
- 单线程事件处理：调度事件串行处理，大规模集群下可能成为瓶颈
- 配置复杂度：CS 配置项超过 50 个，队列层级深时配置维护成本高

## 11.3 质量评估

| 评估维度   | 评分     | 说明                       |
|--------|--------|--------------------------|
| 代码结构   | ★★★★☆  | 分层清晰，接口设计好，但部分基类过大       |
| 设计模式运用 | ★★★★★  | 模板方法/策略/组合/观察者/责任链等多模式协作 |
| 可扩展性   | ★★★★☆  | 接口丰富，自定义策略/规则方便          |
| 文档完整度  | ★★★★☆☆ | 核心类有文档，部分内部类缺少注释         |
| 测试覆盖度  | ★★★★☆  | 306个源文件配套大量单元测试          |